UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 pure-extraction test paper (November 2026). Answers are reference answers for the extraction test. They are not official UTM marking schemes.

2. Marking points are reference guidance; check the answer against the source paper and the original figure.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Extraction test: 11 real questions; 7 asset-bearing questions; 0 placeholders.
Questions and answers are for local study use only.

5. Questions are reproduced from the supplied CSC429 papers; figures are cropped from the source page and captions are native text.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Source: CSC429-2019-12, page 2

Reference answer:

a) Inventor 'Y' is Blaise Pascal. The OCR text identifies him as France's celebrated mathematician and physicist who, in 1642, built the first working calculating machine to help his father with tax accounting; this matches Pascal's Pascaline.

b) Another calculating machine from the same generation (17th century, early mechanical calculators) is Wilhelm Schickard's calculating clock (1623) — alternatively Gottfried Wilhelm Leibniz's stepped reckoner (1673). Either is acceptable as a same-generation mechanical calculating machine.

c) The programming language named after Pascal is Pascal.

d) The Pascaline could perform addition and subtraction (directly); it could also support multiplication and division by repeated addition/subtraction. Any TWO of: addition, subtraction (and, by repetition, multiplication and division).

Marking points:

• *a) Correctly identifies Blaise Pascal (1 mark).*

• *b) Names a valid same-generation calculating machine, e.g., Schickard's calculating clock or Leibniz's stepped reckoner (1 mark).*

• *c) States Pascal (1 mark).*

• *d) States two valid operations, e.g., addition and subtraction (2 marks; 1 mark each).*

Note: Figure 1 is a captured portrait from the source page; the surrounding snippet text is native text.

QUESTION A2 (5 marks)

Source: CSC429-2019-06, page 2

Reference answer:

a) Component X is the integrated circuit (IC), also called a microchip/chip: a single complete electronic semiconductor circuit fabricated on a piece of silicon. The OCR description ('single, complete electronic semiconductor circuits contained on pieces of silicon') identifies it as the integrated circuit.

b) Component X (the integrated circuit) was introduced in the third computer generation (1964–1971), the generation characterised by integrated circuits replacing discrete transistors.

c) The integrated circuit was invented by Jack Kilby (Texas Instruments, 1958), with Robert Noyce (Fairchild, 1959) independently developing a practical silicon IC; Jack Kilby is the standard credited inventor.

d) In the third generation, input devices included the keyboard (and, in that era, the mouse, invented by Douglas Engelbart in 1964); output devices included the monitor/VDU (visual display unit) and the line printer. One valid pair: input = keyboard, output = monitor (VDU).

Marking points:

• *a) 1 mark: correctly names component X as integrated circuit (IC)/microchip.*

• *b) 1 mark: states third generation.*

© Hak Cipta Universiti Teknologi MARA

• c) 1 mark: names Jack Kilby (accept Robert Noyce as co-inventor).

• d) 2 marks: 1 mark for a valid third-generation input device (e.g. keyboard, mouse); 1 mark for a valid third-generation output device (e.g. monitor/VDU, printer).

Note: Figure 1 is a captured photo from the source page; the surrounding snippet text is native text.

QUESTION A3 (5 marks)

Source: CSC429-2025-02, page 2

Reference answer:

a) The 40-bit string 0101011100111000111100101100111000011001 is split into five 8-bit bytes: 01010111, 00111000, 11110010, 11001110, 00011001. In hexadecimal these are 57h, 38h, F2h, CEh, 19h, so the byte content is 57 38 F2 CE 19 (hex), i.e. 01010111 00111000 11110010 11001110 00011001 in binary. b) Workflow: convert each hex digit to 4 bits. 2D.5h = 0010 1101 . 0101 and ED.2h = 1110 1101 . 0010. Align the radix points and add: fractional part 0101 + 0010 = 0111 (no carry). Integer part: 0010 1101 + 1110 1101 = 1 0001 1010, i.e. 0001 1010 with a carry out of 1 (2Dh + EDh = 11Ah). Result = 1 0001 1010 . 0111 = 100011010.0111 in binary (11A.7h).

Marking points:

• a) Correctly splits the 40-bit string into five 8-bit groups (1 mark).

• a) Correctly converts each group to hex giving 57 38 F2 CE 19 (1 mark).

• b) Correctly converts both operands to binary: 00101101.0101 and 11101101.0010 (1 mark).

• b) Correctly aligns radix points and adds fractional parts 0101 + 0010 = 0111 (1 mark).

• b) Correctly adds integer parts including carry, giving 1 0001 1010, and states final result 100011010.0111 (1 mark).

Note: Figure 1 is supplied only as OCR text (a 40-bit binary string); it is assumed to be exactly the 40 bits shown, giving five bytes. No unseen arrows or labels are assumed. The answer is a reference solution, not an official UTM marking scheme.

QUESTION A4 (5 marks)

Source: CSC429-2023-07, page 2

Reference answer:

a) Sign-and-magnitude method for $-48_{10} + 68_{10}$: The operands have opposite signs, so subtract the smaller magnitude from the larger magnitude: 68 − 48 = 20. The sign of the result is the sign of the operand with the larger magnitude, i.e. positive (68 is positive). Therefore $-48_{10} + 68_{10} = +20_{10}$. In sign-and-magnitude representation (using 8 bits for illustration), -48 = 1 0110000 and +68 = 0 1000100; the magnitude subtraction gives 010100 (20) and the sign bit is 0, yielding 0 0010100 = +20.

b) 1's and 2's complement method for $-96_{10} - 15_{10}$: Use 8-bit representation. +96 = 01100000, +15 = 00001111. In 1's complement, -96 = 10011111 and -15 = 11110000. In 2's complement, -96 = 10100000 and -15 = 11110001. The operation -96 - 15 is equivalent to (-96) + (-15). 1's complement addition: 10011111 + 11110000 = 1 10001111; add end-around carry: 10001111 + 1 = 10010000, which is the 1's complement representation of -111. 2's complement addition: 10100000 + 11110001 = 1 10010001; discard the final carry, giving 10010001, which is the 2's complement representation of -111. Thus $-96_{10} - 15_{10} = -111_{10}$.

Marking points:

© Hak Cipta Universiti Teknologi MARA

• a) Correctly identifies opposite signs and subtracts smaller magnitude from larger magnitude (68 − 48 = 20) [1 mark]

• a) Assigns correct positive sign from the larger-magnitude operand and states final result $+20_{10}$ [1 mark]

• b) Converts +96 and +15 to 8-bit binary and obtains their 1's and 2's complement negative forms correctly [1 mark]

• b) Performs 1's complement addition with end-around carry and interprets result as -111_{10} [1 mark]

• b) Performs 2's complement addition, discards final carry, and interprets result as -111_{10} [1 mark]

Note: No figure or table was supplied; the 8-bit word length is assumed for illustration in part (b) and for the binary representation in part (a). The question does not specify a fixed word size, so any consistent word length that can represent the operands and results is acceptable. This is a reference answer, not an official UTM marking scheme.

QUESTION A5 (10 marks)

Source: CSC429-2026-02, page 3

Reference answer:

a) An interrupt is a signal (hardware or software) sent to the CPU that temporarily suspends the currently executing program, saves its state (context), and transfers control to a dedicated interrupt service routine (ISR)/handler; after the handler completes, the saved context is restored and the interrupted program resumes. It allows the CPU to respond to asynchronous events (I/O completion, timers, errors) without polling.

b) Interrupt W has the highest priority. In a nested-interrupt figure the handler that is entered first and is never preempted by any other handler (all others nest inside it) is the highest-priority one; the OCR shows W as the outermost handler, so W is assumed highest priority.

c) Interrupt Z has the lowest priority. It is the innermost handler in the nesting (it is entered last and is preempted by none, i.e., it can be interrupted by all others), which corresponds to the lowest priority; this is assumed from the OCR ordering W > X > Y > Z.

d) With the defined (fixed) priority concept, a higher-priority interrupt preempts a lower-priority one, so handlers complete in reverse order of priority: Z completes first, then Y, then X, then W. Completion sequence: Z, Y, X, W.

e) With the disable-nested-interrupt concept, once a handler starts, further interrupts are disabled until it finishes, so handlers run to completion in the order they arrive. Assuming arrival order W, X, Y, Z (as shown nested), the completion sequence is W, X, Y, Z.

Marking points:

• a) 1 mark: definition as a signal that suspends the current program and transfers control to an ISR; 1 mark: mention of saving/restoring context (or asynchronous event handling).

• b) 1 mark: identifies W; 1 mark: justification that it is the outermost/non-preempted handler (highest priority).

• c) 1 mark: identifies Z; 1 mark: justification that it is the innermost handler preempted by all others (lowest priority).

• d) 2 marks: correct completion order Z, Y, X, W (reverse priority); award partial if order partly correct.

• e) 2 marks: correct completion order W, X, Y, Z (arrival order, no nesting); award partial if order partly correct.

Note: Figure 2 is supplied only as OCR text ('User program; Interrupt handler W; X; Y; Z') with no arrows or explicit priority labels. Assumption: the nesting order is W outermost to Z innermost, giving priority W > X > Y > Z and arrival order W, X, Y, Z. If the actual figure shows different nesting/arrows, the priority ranking and sequences in (b)-(e) must be re-derived accordingly. This is a reference answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION A6 (10 marks)

Source: CSC429-2024-07, page 3

Reference answer:

a) The word size is 32 bits. Justification: the word size of a processor is conventionally defined by the width of its data bus, since the data bus determines how many bits can be transferred to/from the CPU in a single transfer. With a 32-bit data bus, one word = 32 bits.

b) The register size is 32 bits. Justification: general-purpose registers must be able to hold a full word so that a single bus transfer can load/store a complete operand; register width therefore matches the data bus/word size (32 bits). (The 8-bit address bus limits addressable memory to 2^8 = 256 locations; it does not set register width.)

c) System interconnection (the bus) is the only path for transferring data, addresses and control signals between CPU, memory and I/O. If it is blocked, no machine cycle can complete: during the fetch cycle the CPU cannot place the address on the address bus or read the instruction from memory, so the PC cannot be updated and instruction fetch stalls; during the execute cycle operands cannot be read from memory/I/O and results cannot be written back, and control signals (read/write, clock, interrupt) cannot reach the target units. The CPU would effectively hang or repeatedly retry/time-out, and the system cannot progress through fetch-decode-execute; any transfer in progress is aborted and the machine cycle cannot terminate normally.

d) CPU interconnection (the internal CPU bus/control paths linking the ALU, registers, control unit and internal buses) carries data between registers and the ALU, supplies operands and stores results, and distributes control signals from the control unit that select register transfers and ALU operations. It thus enables the internal steps of each machine cycle (e.g., register transfer, ALU operation, PC increment) to be sequenced and coordinated.

Marking points:

• *a) States word size = 32 bits (1 mark); justifies via data bus width / bits transferred per transfer (1 mark).*

• *b) States register size = 32 bits (1 mark); justifies that registers hold a full word matching the data bus (1 mark).*

• *c) Explains bus carries data/address/control between CPU, memory and I/O (1 mark); fetch cycle cannot fetch instruction / PC not updated (1 mark); execute cycle cannot read operands or write results (1 mark); machine cycle stalls/hangs, no normal completion (1 mark).*

• *d) Role of CPU interconnection: internal data path between registers and ALU (1 mark); carries control signals sequencing internal operations/machine-cycle steps (1 mark).*

Note: No figure/table was supplied (question is text-only), so no unseen arrows or labels are assumed. Standard textbook convention is used: word size = data bus width; register size = word size. The 8-bit address bus gives 256 addressable locations and the 4-bit control bus gives 16 control-signal combinations; these are context only and do not change (a) or (b). This is a reference answer, not an official UTM marking scheme.

QUESTION A7 (10 marks)

Source: CSC429-2023-02, page 5

Reference answer:

a) MIMD with shared memory: Draw N processing units (PUs), each with its own control unit (CU) so each has an independent instruction stream (IS) entering its PU. All PUs are connected through a single shared memory

© Hak Cipta Universiti Teknologi MARA

unit (MU) via an interconnection network (e.g., a common bus or crossbar). Each PU has its own data stream (DS) to/from the shared MU; label the IS arrows into each CU/PU and the DS arrows between each PU and the shared MU. MIMD with distributed memory: Draw N independent nodes, each node containing its own CU, PU and local MU (private memory). Each node has its own IS into its CU/PU and its own DS between its PU and its local MU. The nodes are connected to one another by an interconnection network (e.g., a network/message-passing link); label the IS and DS within each node and the inter-node communication links. b) Examples: MIMD with shared memory — a multicore processor/server (e.g., symmetric multiprocessor, SMP) where cores share main memory; MIMD with distributed memory — a cluster of workstations or a massively parallel processor (MPP) such as a Beowulf cluster. c) Advantage: distributed memory is highly scalable because each processor has its own private memory, so adding processors adds memory bandwidth and capacity without contention on a single shared memory/bus; it also avoids the shared-memory bottleneck and allows processors to operate independently (no cache-coherence problem on a single bus).

Marking points:

• *a) i) Correct MIMD shared-memory diagram: multiple PUs each with own CU/IS, all connected to one shared MU via interconnection network, with DS labelled (3 marks)*

• *a) ii) Correct MIMD distributed-memory diagram: multiple nodes each with own CU, PU and local MU, IS and DS labelled within each node, and interconnection network between nodes (3 marks)*

• *b) One valid example for shared-memory MIMD (e.g., SMP/multicore) (1 mark)*

• *b) One valid example for distributed-memory MIMD (e.g., cluster/MPP) (1 mark)*

• *c) One valid advantage of distributed memory, e.g., scalability/no shared-memory bottleneck (2 marks)*

Note: Figure 3 is captured from the source page. The answer describes the standard SISD-to-MIMD extension because the question asks the student to draw.

QUESTION A8 (10 marks)

Source: CSC429-2023-07, page 5

Reference answer:

a) A symmetric multiprocessor (SMP) organization redrawn from Figure 2: keep the same set of processors and main memories, but replace the NUMA's distributed, non-uniform access with a single shared main memory reached through one common interconnect (bus or switch). All processors connect to the same interconnect and therefore to the same main memory, so every processor has equal access time to all of memory. In the redrawn figure, draw N processors on one side, each attached to a common interconnect, and a single shared main memory (or a set of memory modules) on the other side of that interconnect; no processor has a private main memory of its own. (Assumption: the OCR of Figure 2 shows processors each paired with a 'Main Memory' plus an interconnect, i.e. a NUMA with distributed memory; the SMP redraw collapses these into one shared memory on a common interconnect.)

b) Two examples of tightly coupled MIMD parallel processor architectures: (1) Symmetric Multiprocessor (SMP) / shared-memory multiprocessor, in which all processors share one main memory and communicate through shared variables over a common interconnect. (2) Multicomputer/cluster or, more precisely for tightly coupled MIMD, a shared-memory multiprocessor with a crossbar/switch interconnect (e.g. a UMA multiprocessor); alternatively a tightly coupled MIMD system built as a bus-based SMP or a switch-based SMP. (Accept any two valid tightly coupled MIMD examples such as SMP and NUMA/shared-memory

© Hak Cipta Universiti Teknologi MARA

multiprocessor, provided they are tightly coupled MIMD.)

c) Two disadvantages of tightly coupled MIMD parallel processor architectures: (1) Limited scalability and interconnect contention — because all processors share a single memory/interconnect, adding more processors increases contention and saturates the bus/switch, so performance does not scale well and access latency grows. (2) Memory access conflicts/coherence overhead and higher cost/complexity — simultaneous access to shared data requires synchronization and cache-coherence mechanisms, causing memory conflicts and communication overhead, and the shared memory and interconnect make the system more expensive and harder to design and expand.

Marking points:

• *a) 2 marks: 1 mark for showing multiple processors connected to a common interconnect; 1 mark for a single shared main memory (uniform access) replacing the per-processor distributed memories of the NUMA figure.*

• *b) 4 marks: 2 marks for each correct tightly coupled MIMD example (e.g. SMP/shared-memory multiprocessor; NUMA/shared-memory multiprocessor or switch-based SMP), up to 4 marks for two examples.*

• *c) 4 marks: 2 marks per valid disadvantage with a brief explanation (e.g. limited scalability/interconnect contention; memory conflicts/coherence and synchronization overhead; cost/complexity), up to 4 marks for two disadvantages.*

Note: Figure 2 is supplied only as OCR text ('1-1 Memory I Processor Main M?oory N Processor Main Mernory 2 interconnect'), so the redraw in (a) assumes the figure shows N processors each with its own main memory linked by an interconnect (distributed-memory NUMA). No unseen arrows or labels are invented; the SMP redraw is described in words and should be drawn as processors on a common interconnect to one shared main memory. This is a reference answer, not an official UTM marking scheme.

QUESTION A9 (10 marks)

Source: CSC429-2025-02, page 4

Reference answer:

a) In interrupt-driven I/O, the CPU issues the I/O command and then continues executing other instructions; when the device completes its operation, it raises an interrupt, causing the CPU to suspend its current program, save state, and branch to an interrupt service routine (ISR) to transfer data or handle the device. Thus the CPU is involved in every data transfer (or at least in servicing each interrupt) and must execute the ISR. In DMA, the CPU programs the DMA controller with the starting address, word count, and direction, then the DMA controller transfers data directly between the device and memory without CPU intervention; the CPU is interrupted only once per block transfer (or at completion), so it is largely uninvolved during the actual data movement. Key difference: interrupt-driven I/O requires CPU involvement per transfer/interrupt, whereas DMA offloads bulk data movement to a dedicated controller, minimising CPU overhead.

b) An interrupt is a signal (hardware or software) that causes the CPU to temporarily suspend the execution of the current program, save the current context, and transfer control to a specific interrupt service routine (ISR) to handle an event; after servicing, the CPU restores the saved context and resumes the interrupted program.

c) During a context switch, the operating system saves the state (context) of the currently running process—typically the program counter, CPU registers, and process status word—into its process control block (PCB). It then selects the next process to run, loads that process's saved context from its PCB into the CPU registers, and resumes execution of the selected process. This may also involve updating scheduling data structures and memory-management registers.

© Hak Cipta Universiti Teknologi MARA

d) The benefit of a context switch is that it enables multitasking and efficient CPU utilisation: by switching between processes, the OS can share the CPU among multiple processes, allow a process to continue after an I/O wait or interrupt, and enforce scheduling policies (e.g., time-sharing, priority), thereby improving system responsiveness and throughput.

Marking points:

• *a) 4 marks: 2 marks for correct description of CPU involvement in interrupt-driven I/O (CPU issues command, continues, then services interrupt/ISR per transfer); 2 marks for correct description of DMA (CPU programs DMA controller, DMA transfers data directly to/from memory, CPU interrupted only at completion/block level). Award partial marks for partial contrast.*

• *b) 2 marks: 1 mark for stating interrupt is a signal/event that suspends current program; 1 mark for transfer to ISR/handler and later resume (or equivalent).*

• *c) 2 marks: 1 mark for saving current process context (PC, registers, status) to PCB; 1 mark for loading next process context and resuming it.*

• *d) 2 marks: 1 mark for enabling multitasking/CPU sharing among processes; 1 mark for improving responsiveness/throughput/utilization or allowing process to resume after wait.*

Note: No figure or table was supplied; the question is text-only. The answer assumes standard CSC429 course terminology. This is a reference answer, not an official UTM marking scheme.

QUESTION A10 (10 marks)

Source: CSC429-2024-07, page 4

Reference answer:

a) The icon in Figure 1 depicts a USB (Universal Serial Bus) interface/port, i.e. a USB connector symbol used to represent the interface to external devices. (Assumption: the figure is supplied only as a caption with empty OCR text, so the identification is based on the stated caption 'Interface to the external devices' and the standard USB icon convention; no unseen labels or arrows are assumed.)

b) A hot connection (hot plugging/hot swapping) is the ability to connect or disconnect an external device to/from a running computer system without powering down or restarting the system; the interface and operating system detect and configure the device dynamically while the system continues operating.

c) The three I/O techniques are: (1) Programmed I/O (polled I/O): the CPU issues an I/O command to the device controller and then repeatedly checks (polls) the device status register until the operation completes, transferring data through the CPU registers; the CPU is fully involved and busy-waits, wasting cycles while the slow device operates. (2) Interrupt-driven I/O: the CPU issues the I/O command and continues executing other instructions; when the device completes (or needs service), it raises an interrupt, the CPU suspends its current program, saves context, runs the interrupt service routine to transfer/handle the data, then resumes; CPU involvement occurs only at interrupt time, avoiding busy-waiting but still requiring the CPU to move each data unit. (3) Direct Memory Access (DMA): the CPU programs a DMA controller with the memory address, count and direction, then the DMA controller transfers data directly between the device and main memory without CPU intervention, raising an interrupt only when the whole block is complete; the CPU is involved only at setup and completion, giving the highest efficiency for block transfers.

Marking points:

• *a) Correctly identifies USB (Universal Serial Bus) interface/port from the icon (2 marks).*

© Hak Cipta Universiti Teknologi MARA

• b) Defines hot connection as connecting/disconnecting a device while the system is powered on and running, without restart or shutdown (2 marks).

• c) Names the three techniques: programmed I/O, interrupt-driven I/O, DMA (up to 2 marks).

• c) Explains programmed I/O with CPU polling/busy-wait and full CPU involvement (about 1–1.5 marks).

• c) Explains interrupt-driven I/O with device interrupt, context save/restore and CPU servicing only on interrupt (about 1–1.5 marks).

• c) Explains DMA with CPU setup only, direct device–memory transfer by DMA controller and completion interrupt (about 1–1.5 marks).

• c) Elaborates the degree/nature of CPU involvement in each technique (about 1 mark).

Note: Figure 1 is a captured icon from the source page; the USB identification is based on the source figure and caption.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Source: CSC429-2026-02, page 6

Reference answer:

Assumption: Figure 5 is given only as OCR text, read as z = A + C²/(B·D) − 3A, i.e. z = A + (C*C)/(B*D) − 3*A, with A, B, C, D fixed registers and x, y, z temporary registers; the final result must end in z. Table 1 is taken as the complete instruction set (zero-address: PUSH p, POP p, ADD, SUB, MUL, DIV; one-address: LOAD p, STOR p, ADD p, SUB p, MUL p, DIV p; three-address: ADD (p,q,r), SUB (p,q,r), MUL (p,q,r), DIV (p,q,r), meaning p ← q op r).

(a) Zero-address (stack) program. Evaluate in postfix order: push A; push C; push C; MUL; push B; push D; MUL; DIV; ADD; push 3; push A; MUL; SUB; POP z. Written as instructions:

```
PUSH A
PUSH C
PUSH C
MUL
PUSH B
PUSH D
MUL
DIV
ADD
PUSH 3
PUSH A
MUL
SUB
POP z
```

This computes C*C, then B*D, divides, adds A, computes 3*A, subtracts, and stores to z.

(b) One-address program (accumulator machine). Use the accumulator implicitly and x, y as temporaries:

```
LOAD C
MUL C       ; ACC = C*C
STOR x      ; x = C*C
LOAD B
MUL D       ; ACC = B*D
STOR y      ; y = B*D
LOAD x
DIV y       ; ACC = C*C/(B*D)
ADD A       ; ACC = A + C*C/(B*D)
STOR x      ; x = partial result
LOAD A
MUL 3       ; ACC = 3*A  (literal 3 as operand)
```

© Hak Cipta Universiti Teknologi MARA

```
STOR y      ; y = 3*A
LOAD x
SUB y       ; ACC = A + C*C/(B*D) - 3*A
STOR z      ; z = final answer
```

(c) Three-address program. Each instruction is p ← q op r; use x, y, z as temporaries and finish in z:

```
MUL (x, C, C)     ; x = C*C
MUL (y, B, D)     ; y = B*D
DIV (x, x, y)     ; x = C*C/(B*D)
ADD (z, A, x)     ; z = A + C*C/(B*D)
MUL (y, 3, A)     ; y = 3*A
SUB (z, z, y)     ; z = A + C*C/(B*D) - 3*A
```

Final answer is in z as required.

Marking points:

• *Correct interpretation of Figure 5 as z = A + C^2/(B*D) - 3A (with stated assumption)*

• *Zero-address: correct postfix/RPN ordering of operands and operators (C C MUL, B D MUL, DIV, ADD, 3 A MUL, SUB)*

• *Zero-address: correct use of PUSH/POP and final POP z (2 marks)*

• *One-address: correct use of LOAD/STOR with accumulator and temporaries x, y*

• *One-address: correct operator sequence MUL, DIV, ADD, MUL, SUB and final STOR z (3 marks)*

• *Three-address: correct three-operand form p <- q op r for MUL, DIV, ADD, SUB*

• *Three-address: correct temporary allocation and final result in z (3 marks)*

• *Correct assembly notation/syntax consistent with Table 1 throughout (2 marks)*

• *Overall logical correctness and no undefined registers used (2 marks)*

*Note: Figure 5 was supplied only as OCR text 'z=A+ C2/B*D-3A'; it is assumed to mean z = A + C^2/(B*D) - 3A with standard operator precedence (exponentiation before multiplication/division, then addition/subtraction). If the intended reading were z = A + C^2/B*D - 3A evaluated strictly left-to-right for * and /, the DIV/MUL order would change; the stated assumption is used. Table 1 is assumed complete; literal constants (3) are assumed usable as operands in one- and three-address forms. This is a reference answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA